The Universal Elimination Method

Producing Structural Findings Through Exhaustive Enumeration and Constraint-Based Elimination

Registry: [HOWL-INFO-18-2026]

Series Path: [HOWL-INFO-14-2026] → [HOWL-INFO-18-2026]

Date: November 2026

DOI: 10.5281/zenodo.zzz

Domain: Information Theory / Information Processing Theory

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

---

1. The Problem: Structure Exists Before You See It

Every domain has structure. The equations of fluid mechanics have structure. The behavioral patterns of a software system have structure. The propagation mechanisms available in biological tissue have structure. The substances a human body can ingest have structure. The structure is present in the elements of the domain and in the relationships between them. The structure is there whether or not anyone has identified it.

The structure is not visible for one of three reasons.

The first reason is that the elements have not been listed completely. A practitioner who knows a domain deeply may still work with a partial picture because they have never written down every element in one place. Expertise dissolves individual elements to zero cost — the expert handles each element effortlessly — but expertise does not automatically produce the complete enumeration that reveals the pattern across all elements. A radiologist who reads chest films instantly has dissolved the reading of each film. That dissolution does not produce a list of every possible finding across every imaging modality. The expert knows each element. The expert does not necessarily see the structure that connects them, because the structure is a property of the complete set, not of any individual element.

The second reason is that the constraint that separates essential from inessential has not been stated. The elements are all present, but they are mixed together — survivors and non-survivors in one undifferentiated list. Without a constraint that tests each element, the list contains everything and reveals nothing. A list of every physical mechanism that could carry a signal through biological tissue is not a finding. It is an inventory. The finding emerges when a constraint — in this case, the timing inequality that the mechanism must satisfy — separates the mechanisms that can do the job from the mechanisms that cannot.

The third reason is that the surviving elements have not been named. The enumeration was performed. The constraint was applied. The non-survivors were removed. But the result was left as an unlabeled residual — a list of things that passed, sitting in a notebook or a spreadsheet, never given a name that makes the structure communicable. An unnamed finding does not propagate. It is not referenced by others. It is not built upon. It sits where it was produced and contributes nothing beyond the investigator's own understanding. The name is what converts a private observation into a public finding.

These three failures are independent. A practitioner can fail at any one while succeeding at the other two. The method described in this paper addresses all three systematically: it produces the complete enumeration, applies the constraint, and names the result.

The standard approach to finding structure in a domain is hypothesis-driven research. The investigator proposes an explanation, designs a test, collects data, and evaluates whether the data supports the explanation. This approach works. It has produced the majority of scientific knowledge. But it requires the investigator to already suspect what the structure is — to form a hypothesis before collecting evidence. The hypothesis comes from intuition, prior work, analogy, or luck. The method described in this paper works before the hypothesis exists. It produces the structural finding that a hypothesis would need to explain.

The method does not require the domain to be unfamiliar. A practitioner can apply it to their own specialty — the domain they know best — and discover structure they missed despite years of expertise. This is not a failure of the expert. It is a consequence of how expertise works. Expertise optimizes for handling individual elements efficiently. The method optimizes for seeing the pattern across all elements simultaneously. These are different operations, and excellence at one does not produce the other.

---

2. The Method in One Sentence

Enumerate every element in a domain, eliminate what fails an inarguable constraint, name what survives.

Each word of this sentence is load-bearing. The remainder of this section defines each term precisely.

Enumerate means list exhaustively. Not sample. Not estimate. Not list the obvious ones. List every element. The enumeration must be complete because the method's power comes from the claim that nothing survives except what the constraint permits. If elements are missing from the enumeration, that claim is void.

Every element means the atomic unit the investigation operates on. What counts as an element depends on the question being asked. In one investigation, the elements are physical propagation mechanisms in biological tissue. In another, they are terms in published equations. In another, they are ingestible substances. In another, they are state changes in a software system. In another, they are units of information activity. The element type is determined by the question. The question determines what gets enumerated.

Inarguable constraint means a bound that does not depend on the investigator's judgment, interpretation, or preference. An arithmetic inequality is inarguable — a number either exceeds a bound or it does not, and any reader can verify the comparison. A physical measurement is inarguable — the measured propagation speed of a myelinated nerve fiber is what independent laboratories have measured it to be. A logical requirement is inarguable — a specification either expresses a known behavior or it does not. A body response is inarguable — a substance either destabilizes homeostatic regulation under sustained intake or it does not, and the response is observable. The constraint must be the kind of thing a reader can check without trusting the investigator.

Eliminate means apply the constraint to every element. Each element either passes or fails. The elimination is binary. There is no partial pass. There is no judgment call. The constraint does the work. The investigator records the result. If the investigator finds themselves making judgment calls during elimination, the constraint is not inarguable and must be revisited.

Name what survives means assign a specific label to the structure the elimination revealed. Before the naming, the survivors are an unlabeled residual. After the naming, they are an identified finding that can be communicated, referenced, tested, and built upon by others. The name is what makes the finding real outside the investigator's own mind.

---

3. The Five Steps

The single sentence unpacks into five operational steps. Each step has a specific input, a specific output, a specific failure mode when skipped, and a specific relationship to the steps around it.

Step 1: Select the question and identify the element type

The investigator chooses what to investigate. The question determines what the elements are.

"What can carry a coordination signal through biological tissue?" makes the elements physical propagation mechanisms — myelinated axons, gap junctions, bone conduction, hydraulic waves, fascial transmission, piezoelectric signaling, chemical diffusion, electromagnetic field propagation. Each mechanism is one element. Each can be tested against a timing constraint.

"What is the common structure in equations involving circular cross-sections?" makes the elements terms in published formulas from whatever departments use circular cross-sections — fluid mechanics, electromagnetism, optics, thermal physics, aerodynamics, antenna theory. Each equation is one element. Each can be tested by algebraic decomposition.

"Is this substance food?" makes the elements ingestible substances — meats, grains, processed foods, pure sugar, salt, water, poisons. Each substance is one element. Each can

be tested by body response under specific intake patterns.

"What state changes can this system undergo?" makes the elements events in the software system — every transition, every action, every response the system can produce. Each event is one element. Each can be tested against the one-sentence rule: can this event be explained in one sentence as a single bounded state change?

The element type must be the thing the constraint can test. If the constraint is a timing inequality, the elements must have measurable propagation speeds. If the constraint is an algebraic decomposition, the elements must be terms that can be decomposed. If the constraint is a body-response evaluation, the elements must be substances whose body response is observable. Choosing the wrong element type produces an enumeration that cannot answer the question.

Failure mode: wrong element type. The enumeration is complete and the constraint is inarguable, but the constraint cannot test the elements because they are the wrong kind of thing. The elimination produces no result. The correction is to return to Step 1 and re-examine the question until the element type and the constraint type align.

Step 2: Enumerate exhaustively

List every element. The list must be complete.

Completeness is what gives the method its power. When the enumeration is complete, the elimination proves that nothing outside the surviving set can satisfy the constraint. This is a proof by exhaustion — every candidate was tested, and only the survivors passed. If the enumeration is incomplete, the proof is void. An untested element might have passed, and the finding is wrong.

Exhaustive enumeration requires generous inclusion. When uncertain whether something belongs in the enumeration, include it. An element that is included and then eliminated costs one check — one application of the constraint, one line in the elimination record. An element that is excluded and later discovered to survive the constraint invalidates the entire finding. The cost of over-inclusion is trivial. The cost of under-inclusion is the finding.

Exhaustive enumeration is the most labor-intensive step and the one most often shortcut. The shortcut takes many forms. "We only need to consider the main candidates." "That one is obviously irrelevant." "Nobody uses that mechanism anymore." "That substance is not considered food." Every one of these shortcuts removes an element from the enumeration without applying the constraint to it. Every one risks removing a survivor.

The discipline is absolute: include everything. Let the constraint decide what belongs.

Failure mode: incomplete enumeration. A missing element that would have survived the constraint changes the finding. The finding is wrong not because the analysis failed but because the input was incomplete. This is the most common failure of the method and the hardest to detect because the investigator does not know what they missed. The correction is ongoing: when a new element is discovered that was not in the original enumeration,

add it, apply the constraint, and update the finding. The method is permanently revisable because the enumeration is permanently extensible.

Step 3: State the constraint

The constraint is the mechanism that does the work. It must satisfy two requirements: it must be inarguable, and it must be stated before the elimination begins.

Inarguable means the constraint's application to any element produces a result that any competent reader can verify independently. "This mechanism's round-trip transit time over 100 centimeters, including transduction delays, is 18.1 milliseconds" is verifiable by anyone with the measured propagation speed and the distance. "This equation decomposes into a driving term times π /4 times the bounding area squared times a domain-specific factor" is verifiable by anyone who can do the algebra. "This substance destabilizes blood glucose regulation under daily intake for twenty years" is verifiable by anyone who can read the clinical evidence. The investigator's opinion does not determine the result. The arithmetic, the algebra, or the evidence determines it.

Stated before the elimination means the constraint is fixed before the results are examined. This prevents fitting — adjusting the constraint after seeing which elements survive to produce a desired outcome. Fitting is the most dangerous methodological error because the resulting finding looks exactly like a legitimate finding. The constraint appears inarguable. The elimination appears mechanical. But the constraint was chosen to produce the result, and the finding is circular. The defense against fitting is simple: state the constraint in writing before performing the elimination, and do not change it afterward. If the constraint turns out to be wrong, state a new constraint and perform a new elimination. Do not adjust the original.

Where the exact value of the constraint is uncertain, use the most generous value that still produces a meaningful test. This is the generous constraint, and it deserves its own treatment in the next section.

Failure mode: debatable constraint. The constraint depends on interpretation, the elimination depends on interpretation, and the finding is arguable rather than structural. The correction is to replace the debatable constraint with an inarguable one. If no inarguable constraint can be stated for the question, the question may not be amenable to the UEM, and other methods are needed.

Failure mode: fitted constraint. The constraint was stated or adjusted after the results were known. The finding is circular. The correction is to discard the finding, state the constraint in advance, and re-eliminate from scratch.

Step 4: Eliminate

Apply the constraint to every element in the enumeration. For each element, the result is pass or fail. Record the result and the specific reason.

The elimination record is part of the finding. It is not an intermediate step that can be discarded after the survivors are identified. The record shows what was tested, what failed,

and why. A reader who disagrees with a specific elimination can check the constraint against the element and determine independently whether the elimination is correct. This makes the finding auditable. Auditability is what separates a structural finding from an assertion.

The elimination is mechanical. For each element: does it satisfy the constraint? Yes or no. There is no "almost passes." There is no "passes under certain assumptions." There is no "would pass if we made an adjustment." The constraint is binary. The element passes or fails. If the investigator finds themselves wanting to create intermediate categories during elimination, the constraint is not sufficiently specific and Step 3 must be revisited.

Failure mode: subjective elimination. The constraint was inarguable in statement but arguable in application — the constraint said one thing, but applying it to a specific element required a judgment call. This means the constraint was not specific enough. Return to Step 3 and tighten the constraint until application is mechanical for every element in the enumeration.

Step 5: Name what survives

The surviving elements are the finding. Name them.

The name makes the structure visible. Before the name, the finding is a set of elements that passed a test — a residual list with no identity. After the name, it is a concept. It can be stated in a sentence. It can be referenced by others. It can be tested against new evidence. It can be built upon.

Naming sometimes reveals that the surviving element already existed under different names in different contexts. The elimination removed the material between those contexts, and the naming identified what was underneath — a single structural element with multiple labels. This is not a discovery that the element exists. It is a discovery that the labels refer to the same thing. The element was always there. The connection between the labels was not.

Naming sometimes reveals that the surviving element is genuinely new — no prior name exists because no one had performed the enumeration and elimination that would have revealed it. In this case the name is a contribution to the domain's vocabulary. It gives practitioners a word for something they may have sensed but could not articulate because it had no name.

Naming sometimes reveals that the surviving set is empty — every element failed the constraint. An empty surviving set is a finding. It means the domain does not contain what the question was looking for, at least within the constraint that was applied. If the constraint is inarguable, the empty result is a definitive negative finding: the thing does not exist in this domain. Definitive negatives are among the most valuable findings the method produces, because they redirect effort away from searches that cannot succeed.

Failure mode: unnamed survivors. The elimination was performed correctly but the investigator did not name the result. The finding exists as a private observation. It does

not propagate. It is not built upon. Naming is not a cosmetic step. It is what converts the investigation's output into a contribution.

---

4. The Generous Table

Some constraints have uncertain exact values. The measured propagation speed of a mechanism in biological tissue is known to a range, not a point. The correction frequency required by a specific motor task is an estimate derived from mechanical analysis, not a direct measurement. The threshold at which a substance transitions from occasional tolerance to chronic destabilization varies by individual.

When the constraint's exact value is uncertain, the investigator faces a risk: a reader can dismiss the finding by arguing that the real value is different from the value used. "But myelinated axons might be faster than 120 meters per second in some undiscovered configuration." "But the correction frequency for running might be lower than you estimated." "But some people might tolerate daily sugar intake without destabilization." Each objection questions the constraint and, by extension, the elimination and the finding.

The generous table is the preemptive answer to every such objection. It performs the elimination at multiple constraint levels: the measured value (what has been observed), a generous value (the measured value shifted in the opponent's favor by a meaningful factor, typically $2\times$), and an extreme value (the measured value shifted further, typically $4\times$, with no empirical basis for the shift).

The generous values have no empirical support. They are granted purely for the sake of argument. The investigator is saying: I will give you twice the speed. I will halve the transduction delay. I will double the tolerance threshold. I will do this with no evidence that any of these enhancements exist. And the finding still holds.

If the finding holds at $4\times$ extreme — where every parameter has been shifted maximally in the opponent's favor, with no empirical basis, across every element simultaneously — the finding is robust against any future discovery that shifts the constraint within the generous range. The objection "but the real value might be different" has been answered in advance, for every plausible version of "different."

The generous table is not a concession to weakness in the finding. It is a demonstration of strength. A finding that holds only at measured values is defensible. A finding that holds at $4\times$ extreme is nearly impossible to dismiss, because the dismissal requires arguing that the real values exceed even the most generous assumptions — assumptions the investigator granted freely.

Generosity is appropriate when the constraint's exact value is uncertain or debatable. Generosity is unnecessary when the constraint is exact by definition. The speed of light, an SI constant, a mathematical identity, a logical requirement — these do not have uncertain values. Overprovision on an exact constraint adds nothing. The generous table is a tool for constraints with ranges, not for constraints with exact values.

---

5. The Demand Matrix

The demand matrix is the tool that makes the elimination visible, readable, and communicable in a single table.

The matrix is constructed as follows. Rows are elements from the enumeration. Columns are constraint levels — measured, generous, extreme. Each cell contains the result: PASS, FAIL, or MARGINAL (where the element's value falls within measurement uncertainty of the constraint boundary).

The matrix is the entire finding in one table. A reader who sees only the matrix and nothing else can reconstruct the finding: what was tested, what survived, and how robust each survival is.

The matrix has two reading directions.

Read across a row to see how one element performs at every constraint level. An element that passes at all levels is robust — it survives regardless of how the constraint is set. An element that fails at all levels is definitively eliminated — no plausible shift in the constraint rescues it. An element that passes at extreme but fails at measured is in the uncertain zone — it survives only under the most generous assumptions.

Read down a column to see which elements survive at one constraint level. The surviving set at measured values is the finding under current knowledge. The surviving set at extreme values is the finding under maximum generosity. The difference between these two sets is the uncertainty zone — elements whose survival depends on undiscovered enhancements.

The demand gradient is the pattern that emerges across columns or across groups of rows. If more elements fail as the constraint tightens, the constraint is discriminating — it identifies a real boundary in the domain. If all elements pass at every level, the constraint is not differentiating and the method has found no structure at this constraint. If all elements fail at every level, the finding is that nothing in the domain satisfies the requirement.

The gradient becomes particularly informative when the rows are grouped by demand level. Group the elements by how demanding the task or context is — from least demanding to most demanding. If the pattern of failure increases with demand level, the constraint is identifying a demand-specific boundary. Elements that pass for easy tasks but fail for hard tasks reveal the point where a mechanism stops being sufficient. This demand-dependent failure pattern is the signature of a real structural boundary.

The matrix works within a single domain. It does not require cross-domain comparison. A practitioner investigating the timing requirements of their own system can build a demand matrix entirely within their own domain and read the structural finding directly from the table.

---

6. Within-Domain Application

The UEM does not require crossing domain boundaries. The most common and most immediately useful application is within a single domain where the practitioner already works. Three examples demonstrate this.

Example A: Software system specification

A practitioner asks: what state changes can this system undergo?

The elements are events — every transition, action, response, and state change the system can produce. The constraint is the one-sentence test: can this event be explained in one sentence as a single, bounded state change? If not, it conceals two or more events and must be split into separate elements. If two events always fire together with identical data, they are a single event and must be merged.

The enumeration produces a flat list of every event in the system. Every verb-noun pair in the specification is a candidate. "The kernel mounts the root filesystem" produces `Kernel_Root_Filesystem_Mounted`. "The user enters credentials and is authenticated" produces two events — `User_Credentials_Entered` and `User_Authenticated` — because entering and validating are distinct state changes with independent success and failure modes.

The elimination splits events that fail the one-sentence test and merges events that always co-fire. What survives is a list where every entry is a single bounded state change, explainable in one sentence, independently implementable and testable.

The naming gives each survivor a permanent label. The complete list of named events is the system's specification. Every behavior is named. Every gap — every behavior the system should have but the list does not contain — is visible by inspection. The gap is where a verb-noun pair in the requirements has no corresponding event in the list.

No domain boundary was crossed. The practitioner worked entirely within their own system. The structure revealed — the complete, gap-free event list — was invisible before the enumeration because events were implicit in code, scattered documentation, and team assumptions rather than explicitly listed in one place.

Example B: Information theory

A practitioner asks: what does information do?

The elements are information activities — everything that information undergoes in any system. Information is created, stored, transmitted, received, transformed, interpreted, compressed, expanded, encrypted, decrypted, displayed, discarded. Each activity is one element.

The constraint is coverage by existing theory: does the current mathematical framework provide exact answers for this activity? Claude Shannon's 1948 mathematical theory of communication provides exact answers for the transmission activities — how many bits a

channel can carry, how to encode for efficient transmission, how to overcome noise. The constraint tests each activity against this coverage.

The elimination identifies which activities are covered and which are not. Transmission is covered. Channel capacity is covered. Source encoding is covered. Error correction is covered. But what happens at the endpoints — how a source decides what to transmit, how a destination makes sense of what it receives, how a processor transforms information into action — is not covered. Shannon explicitly excluded it.

What survives the elimination is the uncovered half: information under action, which is processing. Processing requires its own unit (the op — one irreducible transformation by one processor), its own fundamental inequality (total ops times average op duration must not exceed the time budget), and its own mechanisms (dissolution, cascade, reduction). The finding is within information theory. The contribution is the identification of the gap and the framework that fills it.

No domain boundary was crossed. The investigation was entirely within information theory. The structure revealed — the symmetric second half of information theory — was invisible not because it was in another domain but because no one had enumerated every information activity and tested each against the existing theory's coverage.

Example C: Arithmetic representations

A practitioner asks: what representations preserve exact equality under repeated operations?

The elements are arithmetic representation systems — scalar floating point, scalar rational (exact fractions), symbolic computer algebra, interval arithmetic, continued fractions, and any other representation that can be defined.

The constraint is exact equality recovery: start with a value, perform an arbitrary chain of operations, reverse the operations, and test whether the result is exactly the original value. Not approximately. Exactly. No epsilon. No tolerance. The test is structural identity after normalization.

Scalar floating point fails. The mantissa is finite, rounding is silent, and error accumulates. After 200 operations on 1/7 (add 1/13 one hundred times, subtract 1/13 one hundred times), the result differs from 1/7 by approximately 2.78×10^{-16}. The error is small but nonzero. The representation fails the constraint.

Scalar rational (exact fractions) passes for all rational arithmetic. The fraction 1/7 plus 1/13 minus 1/13 returns exactly 1/7 at any chain length. But scalar rational cannot represent structure beyond the single fraction — it has no mechanism for carrying exact unresolved state when a value does not fit cleanly into a given denominator frame.

Symbolic computer algebra passes for exact arithmetic but has no fixed shape — expression trees grow without bound and require rewrite rules to simplify. The representation is not a fixed triple. It is an unbounded structure.

What survives the constraint with both exact recovery and a fixed shape is a representation

that carries three components: a value slot, a denominator frame, and a remainder slot that catches the exact structure the first two slots could not absorb. The remainder is not error. It is the part of the value that lives outside the current frame. The triple [V, D, R] — value, denominator, remainder — passes the constraint at arbitrary chain lengths with zero drift.

The investigation was entirely within arithmetic. No domain boundary was crossed. The finding — that the remainder slot is necessary for exact equality recovery in a fixed-shape representation — was invisible before the enumeration because no one had listed every representation system and tested each against the exact-recovery constraint.

---

7. Cross-Domain Application

When the UEM is applied across domain boundaries, it produces a specific additional output: the discovery that different domains share structure they did not know they shared.

This happens when the enumeration spans multiple domains — when the elements come from different departments, textbooks, or traditions. The constraint tests all elements identically regardless of origin. Elements from different domains that survive the same constraint share the structure the constraint identifies.

Cross-domain findings are not a requirement of the method. They are a consequence of where the investigator draws the enumeration boundary. An investigator who enumerates only within their own domain will find within-domain structure. An investigator who enumerates across domains will find cross-domain structure if it exists. The method is the same in both cases. The scope of the enumeration determines whether cross-domain findings are possible.

Example D: Equations across engineering departments

The investigator asks: what is the common structure in equations that involve circular cross-sections?

The elements are equations from every department that uses circular cross-sections in its formulas. Fluid mechanics has pipe flow equations. Aerodynamics has drag equations. Chemical engineering has orifice flow equations. Electrical engineering has circular plate capacitor equations. Physics has electromagnetic aperture equations. RF engineering has antenna equations. Optics has beam cross-section equations. Thermal physics has radiating surface equations. Pure geometry has the circle area formula. Nine departments. Nine families of equations. Each equation is one element.

The constraint is algebraic decomposition: does the equation decompose into three separable components — a driving term that is domain-specific, a geometric factor that is the same across all equations, and a domain-specific impedance factor? The decomposition is algebraic identity — the rearranged equation is mathematically identical to the original.

Every equation decomposes. The driving term varies: velocity in pipe flow, dynamic

pressure in drag, flux density in electromagnetic transmission, temperature to the fourth power in thermal radiation. The impedance factor varies: friction factor in pipes, drag coefficient in aerodynamics, aperture efficiency in antennas, emissivity in thermal radiation. But the geometric factor is identical in all nine: $\pi/4$ times the square of the diameter. The ratio $\pi/4$ — the area of a circle divided by the area of its bounding square — appears in every equation performing the same structural role: converting the rectilinear bounding area into the circular cross-sectional area.

Nine departments derived this geometric operation independently. Each department gave it its own name: "cross-sectional area," "reference area," "orifice area," "plate area," "aperture area," "effective aperture," "beam area," "surface area." Nine names for one operation. The naming step of the UEM assigns a single name to the surviving invariant: the geometric ratio $\beta = \pi/4$.

The finding is cross-domain because the enumeration spanned nine departments. The structure — one geometric invariant under nine names — was invisible from inside any single department because each department had derived its own equation independently and had no reason to compare its equation's geometric factor with those of other departments.

Example E: Physical carriers across neuroscience and physics

The investigator asks: what physical mechanism can carry a coordination signal through biological tissue fast enough for sub-millisecond motor coordination?

The elements are physical propagation mechanisms, drawn from neurophysiology (myelinated axons, gap junctions), acoustics (bone conduction, hydraulic waves), connective tissue physiology (fascial transmission, piezoelectric signaling), physical chemistry (chemical diffusion), and electrodynamics (electromagnetic field propagation). Eight mechanisms from five disciplines.

The constraint is the control theory timing inequality: the mechanism's round-trip transit time over a given distance, including all transduction delays required to convert between physical domains, must not exceed the correction cycle period demanded by the motor task. This is arithmetic: distance divided by speed, plus transduction costs, compared to the timing bound.

The elimination is performed at three constraint levels: measured speeds, $2\times$ generous speeds, and $4\times$ extreme speeds with halved transduction delays. At measured speeds, only electromagnetic field propagation survives for demanding motor tasks. At $4\times$ extreme — granting every mechanism quadrupled speed and halved delays with no empirical basis — ten of eighteen human motor activities still fail all classical carriers. Only electromagnetic field propagation passes every activity at every constraint level, with six orders of magnitude of headroom.

The finding crosses the boundary between departments that study nerves and departments that study electromagnetic fields. The surviving mechanism comes from electrodynamics, not from neurophysiology. The connection between motor coordination and electromag-

netic field propagation was invisible from inside either department because neurophysiologists enumerate neural mechanisms and electrodynamicists do not study motor timing.

Cross-domain application is a capability of the method. It is not a requirement. The method works identically within a single domain. The cross-domain case produces the additional finding that structure is shared across domains, which is valuable when it occurs. But the method's validity does not depend on crossing boundaries. A within-domain application that produces a clear structural finding is equally valid.

---

8. What the Method Produces

The UEM produces three outputs regardless of whether the application is within-domain or cross-domain.

Structural findings

A structural finding is a statement about what survives elimination. "The only physical mechanism that satisfies the timing inequality for demanding motor tasks is electromagnetic field propagation." "The geometric invariant across nine circular cross-section equations is $\beta = \pi/4$." "The only arithmetic representation with fixed shape and exact equality recovery is the [V, D, R] triple." "A substance is food or not-food depending on the net regulatory effect over the intake pattern."

Each of these statements is checkable by anyone who can read the enumeration and apply the constraint. The finding does not require trust in the investigator. It requires a complete enumeration and an inarguable constraint. If the enumeration is complete and the constraint is inarguable, the finding holds regardless of who performed the investigation, what credentials they hold, or what institution they belong to. The finding stands on its own.

Visible gaps

The enumeration reveals what is missing. An entity group with no events has no lifecycle — it was named but never given anything to do. An event with no constraint fires unconditionally — either intentionally or because no one specified the conditions. A department with no connection to adjacent departments has an unexamined boundary — a gap where cross-domain structure might live.

The gap was invisible before the enumeration because no one had listed everything. After the enumeration, the gap is visible by inspection. You look at the list and see what is absent. The absence is the finding. It tells the investigator where future work lives — where the next enumeration should start, where the next constraint should be tested, where the next structure is likely to be found.

Gaps are among the most valuable outputs of the method because they direct future work rather than merely describing past results. A structural finding says "this is what is here." A gap says "this is what is not here yet, and someone should look."

Falsification boundaries

The elimination record shows exactly what was tested and what would change the finding. If a new element is discovered that survives the constraint, the finding expands — the surviving set is larger than previously known. If a surviving element is shown to fail the constraint under new evidence, the finding contracts — the surviving set is smaller. If the constraint itself is revised by new measurement, the entire elimination can be rerun with the updated constraint.

The finding is permanently open to revision because the method that produced it is permanently re-executable. This is not a weakness. It is the mechanism by which the finding self-corrects. A finding that cannot be revised in light of new evidence is not a structural finding. It is a dogma. The UEM produces findings with built-in revision mechanisms.

The falsification boundary also tells the investigator what would kill the finding. If a classical carrier is discovered that propagates above 1,000 meters per second through soft tissue without transduction delay, the carrier elimination for motor coordination must be revised. If an equation involving a circular cross-section is found that does not contain β as the area-to-bounding-area ratio, the geometric invariant claim must be revised. If a scalar representation is found that achieves exact equality recovery under arbitrary operation chains, the necessity of the remainder slot must be revised. Each of these conditions is specific, testable, and genuinely terminal for the relevant part of the finding. Stating them is not pessimism. It is precision about where the finding's authority ends.

---

9. What the Method Does Not Produce

The UEM does not produce theories. It does not explain why the surviving elements survive. It does not provide the mechanism by which the structure arises. It produces the finding that a theory would need to explain. The theory comes after the finding, not before it. This is the inversion from hypothesis-driven research, where the theory comes first and the evidence tests it. In the UEM, the evidence comes first and the theory explains it.

The UEM does not produce predictions. Predictions come from theories. The UEM produces the structural finding that makes theories possible and constrains what predictions are compatible with the structure. A prediction derived from a UEM finding is a prediction of the theory built on the finding, not a prediction of the method itself.

The UEM does not replace domain expertise. The investigator must know enough about the domain to identify the correct element type (Step 1), produce a complete enumeration (Step 2), and state an inarguable constraint (Step 3). These require domain knowledge. An investigator who does not understand the domain will enumerate the wrong elements, state a weak constraint, or miss elements that should have been included. The method provides the procedure. Domain knowledge provides the inputs.

The UEM does not work on domains where no inarguable constraint exists. If every possible constraint depends on interpretation, then every elimination depends on interpretation, and the finding is arguable rather than structural. Some domains may not have

inarguable constraints at the present time. In those domains, the UEM does not apply, and other methods are needed. The method's scope is limited to domains where at least one inarguable constraint can be stated.

The UEM does not guarantee interesting findings. A complete enumeration against an inarguable constraint may produce a finding that is already known, trivially obvious, or uninteresting. The method guarantees that the finding is structurally sound — that it follows from a complete enumeration and an inarguable constraint. It does not guarantee that the finding matters. Whether a finding matters is a judgment the investigator makes after the method has produced it.

---

10. The Relationship to Information Processing

The UEM is a specific instance of a general structure that appears wherever information is processed.

Processing is bounded by a single inequality: total operations times average operation duration must not exceed the available time budget. Every system that must act on information — a CPU executing instructions, a surgeon deciding where to cut, a pilot classifying a radar contact, a scheduler dispatching processes — operates under this bound. The only lever for staying within the budget is reducing the total number of operations. Everything else — faster hardware, more processors, better algorithms — is a way to reduce the left side of the inequality.

Every element that a processor might need to act on exists, relative to that processor, at one of three cardinalities. Infinity: a population of elements, none selected, the processor unable to act. One: the single element under active operation, the only cardinality at which work happens. Zero: outside the processor's operational boundary, observable but not actionable. Work occurs only at cardinality One. The transition from Infinity to One is called reduction.

Reduction follows a four-stage pipeline. Enumerate: make the population explicit and finite. Filter: eliminate candidates that do not meet relevance criteria. Score: evaluate remaining candidates against weighted considerations. Select: commit the highest-scored candidate as One. The pipeline moves information from unbounded population to actionable element.

The UEM maps directly onto this pipeline. Step 2 (enumerate) is the enumeration stage — it makes the domain's elements explicit and finite. Step 4 (eliminate) is the filter stage — it removes elements that fail the constraint. Step 5 (name) is the selection stage — it identifies the surviving structure as the actionable result. Steps 1 and 3 (select the question and state the constraint) are the setup that determines what gets enumerated and what the filter criterion is.

The UEM is the reduction pipeline applied as an investigation technique. Instead of reducing a run queue to a dispatched process, or a symptom list to a diagnosis, or a candidate

set to a selected option, the UEM reduces a domain from undifferentiated inventory to structural finding.

Through repeated application, a processing chain's cost decreases toward zero. This mechanism — called dissolution — explains expertise, caching, compilation, muscle memory, pattern recognition, and fluency across every domain that processes information. A new driver checks the mirror in six operations. After five years, the check takes zero operations — it happens structurally, without consuming the conscious pipeline. The processing has dissolved into the substrate.

Dissolution applies to the UEM itself. The first time an investigator applies the method, every step requires conscious effort. The element type must be reasoned through. The enumeration must be built deliberately. The constraint must be carefully formulated. The elimination must be performed step by step. After multiple applications across different domains, the method dissolves. The element type identification becomes immediate. The enumeration patterns become familiar. The constraint formulation becomes faster. The elimination becomes mechanical. Each application is cheaper than the last because the method itself has dissolved through use.

This does not mean the findings become trivial. The inputs change with each new domain. The elements are different. The constraint is different. The enumeration is different. What dissolves is the method, not the content. The investigator becomes faster at applying the method while the findings retain their full structural weight.

Dissolution also explains why the method produces increasing output over time. Each domain investigated dissolves into the investigator's working knowledge, providing foundation for the next investigation. A finding in one domain becomes a constraint or an element type in the next. The geometric ratio $\beta = \pi/4$, discovered through the UEM applied to equations, becomes a named element in subsequent investigations of physical measurement anomalies. The carrier elimination result, discovered through the UEM applied to biological tissue, becomes a constraint in subsequent investigations of motor learning and developmental neuroscience. Each finding extends the foundation from which the next investigation begins.

---

11. How to Start

This section provides practical guidance for a reader applying the UEM for the first time.

Pick a domain you work in. Not a domain you want to learn — a domain you already know. The UEM requires domain knowledge for Steps 1 through 3. Applying the method to an unfamiliar domain produces an incomplete enumeration and a weak constraint, both of which void the finding. Start where your expertise is.

Pick a question you cannot currently answer within the domain. Not a question you know the answer to — the method produces findings, and a known answer is not a finding. The question should be something that has resisted resolution through normal methods: a persistent ambiguity, a recurring disagreement, a pattern you can sense but not articulate, a

structure you suspect exists but have not been able to demonstrate. These are the questions the UEM is designed for.

Identify the element type. What are the things that could answer the question? If the question is "why does this system fail under load?" the elements might be resource contention points, or shared dependencies, or communication pathways. If the question is "what is common across these implementations?" the elements might be structural components, or data flows, or design patterns. The element type must be the thing your constraint can test. If you find yourself unsure what the elements are, restate the question more specifically until the elements become clear.

Enumerate them. All of them. Write the list. Use whatever format works — a text file, a spreadsheet, a whiteboard, an enumeration in code. The format does not matter. Completeness does. The list will be longer than you expect because exhaustive enumeration forces you to include elements you would normally skip. Include them. You are building the input to the method, not editing it. Editing comes from the constraint, not from your judgment about what belongs.

State the constraint. What bound separates elements that answer the question from elements that do not? The constraint must be inarguable within your domain. If you cannot state an inarguable constraint, you do not yet understand the question well enough. This is diagnostic, not a failure. Learn more about the question — read, investigate, discuss — until you can state a constraint that any competent practitioner in your domain would accept as non-debatable. Then return and proceed.

Apply the constraint. Build the table. Each element, pass or fail, with the reason. If you have uncertain constraint values, build the generous table: measured, $2\times$ generous, $4\times$ extreme. Read the table. What survived? Is the surviving set smaller than you expected? Does it contain elements you did not anticipate? Does the demand gradient reveal a boundary you did not know existed?

Name the surviving structure. Write it down. The name should be specific enough to communicate the finding and general enough to cover the surviving set. The name is not a label you attach. It is the identity of the structure the elimination revealed.

If the finding is interesting, write it up. The write-up requires three things: the complete enumeration (so readers can verify completeness), the constraint with its justification (so readers can verify it is inarguable), and the elimination record (so readers can verify each pass/fail independently). A write-up containing these three components is auditable. Any reader can re-derive the finding by re-performing the elimination. No trust in the investigator is required.

If the finding is not interesting — if the survivors are already known, or the structure is trivially obvious — you have still reduced the domain. The reduction is permanent knowledge. The enumeration exists. The constraint is stated. The elimination is recorded. Apply the method to the next question. Each application, interesting or not, dissolves one more domain into your working foundation.

---

12. Falsification of the Method

The UEM is falsified if any of the following are demonstrated.

F1: Complete enumeration and inarguable constraint produce no structural differentiation in a domain where the result was already known. A domain is exhaustively enumerated against an inarguable constraint. Every element passes, or every element fails. The universal result was known before the enumeration was performed. The method produced no new information. This does not falsify the method if the universal result was not previously known — discovering that everything passes or everything fails is itself a finding when the result was not expected.

F2: A finding is shown to be an artifact of incomplete enumeration. An element that was not included in the original enumeration is discovered, tested against the constraint, and survives. The element's inclusion would have changed the finding. The method's claim rests on exhaustive enumeration, and incomplete enumeration voided the claim for that specific finding. This does not falsify the method as a procedure — it falsifies the specific finding and demonstrates the consequence of violating Step 2.

F3: A finding is shown to be an artifact of a fitted constraint. The constraint was stated or adjusted after the results were known, producing a circular finding. This does not falsify the method as a procedure — it falsifies the specific finding and demonstrates the consequence of violating Step 3's requirement that the constraint be stated in advance.

F4: A correctly executed UEM produces a finding that is demonstrably wrong. The enumeration was complete. The constraint was inarguable and stated in advance. The elimination was mechanical. The finding — what survived — does not correspond to the domain's actual structure. Not uninteresting. Not trivial. Factually incorrect. The structure the method identified is not the structure the domain has. This would falsify the method's core claim that exhaustive enumeration plus inarguable constraint produces structurally sound findings.

F5: A method is demonstrated that produces the same structural findings without requiring exhaustive enumeration. If the same findings can be reached by sampling, heuristic, or intuition with equal reliability and equal auditability, the UEM's insistence on exhaustive enumeration is unjustified overhead. The enumeration step is the method's primary cost. If the cost is unnecessary for the result, the method is inefficient relative to the alternative.

Each criterion is specific, testable, and would genuinely terminate the relevant claim about the method.

---

13. Conclusion

The Universal Elimination Method is a five-step technique for producing structural findings.

Identify the elements. Enumerate all of them. State an inarguable constraint. Eliminate what fails. Name what survives.

The method works within a single domain, revealing structure that expertise alone does not surface because expertise dissolves individual elements without producing the exhaustive enumeration that reveals the pattern across all elements. The method works across domain boundaries, revealing shared structure that departmental organization prevents anyone from seeing because no department enumerates outside its own scope. The within-domain case and the cross-domain case use the same five steps. The scope of the enumeration determines which kind of finding is possible.

The method produces structural findings, visible gaps, and falsification boundaries. Findings are checkable by anyone who can read the enumeration and apply the constraint. Gaps direct future work. Falsification boundaries make the finding permanently revisable.

The method does not require credentials. It requires a complete enumeration and an inarguable constraint. The finding is auditable by anyone who can verify the enumeration's completeness and the constraint's application to each element. The method is repeatable, falsifiable, and domain-independent.

The method produces findings, not theories. The finding is what survived. The theory is the next paper.

Appendices

Table A: The Five Steps

Step	Name	Input	Output	Failure Mode	Correction
1	Select question and element type	A domain and an unanswered question	The element type that the constraint will test	Wrong element type — constraint cannot test the elements	Restate the question until element type and constraint type align

Step	Name	Input	Output	Failure Mode	Correction
2	Enumerate exhaustively	The element type	A complete list of every element	Incomplete enumeration — a missing element that would survive changes the finding	Include everything; let the constraint decide; extend the enumeration when new elements are discovered
3	State the constraint	Domain knowledge and the question	An inarguable bound stated before elimination	Debatable constraint — the elimination depends on interpretation; Fitted constraint — stated after results are known	Replace debatable constraints with inarguable ones; always state the constraint before performing the elimination
4	Eliminate	The enumeration and the constraint	A pass/fail result for every element with specific reasons	Subjective elimination — judgment calls during application indicate the constraint is not specific enough	Return to Step 3 and tighten the constraint until application is mechanical

Step	Name	Input	Output	Failure Mode	Correction
5	Name what survives	The surviving set	An identified, named structural finding	Unnamed survivors — the finding exists as a private observation and does not propagate	Name the result; the name converts private observation into public finding

---

Table B: Constraint Types

Constraint Type	Definition	When Appropriate	What Makes It Inarguable	Example
Arithmetic inequality	A numerical comparison: value versus bound	When elements have measurable quantities that can be compared to a threshold	The comparison is mechanical — any reader gets the same result	Round-trip transit time versus correction cycle period
Physical measurement	A measured property of a physical system	When elements are physical mechanisms with measured characteristics	The measurement is published by independent laboratories	Propagation speed of myelinated axons: 80-120 m/s
Logical requirement	A structural property that must hold for the element to function	When elements are components of a system with known functional requirements	The requirement follows from the system's definition	A specification must express every known behavior to be complete

Constraint Type	Definition	When Appropriate	What Makes It Inarguable	Example
Body response	An observable physiological response to an input	When elements are substances and the question concerns their effect on the body	The response is measurable through clinical observation	Blood glucose regulation destabilizes under sustained high-glycemic load
Algebraic identity	A mathematical decomposition that preserves the original expression	When elements are equations and the question concerns their structure	The decomposition is verifiable by algebra — the rearranged form equals the original	Equation decomposes into $F \times \beta \times d^2 \times Z$
Timing bound	A maximum allowable time for a process or signal	When elements are mechanisms that must operate within a time window	The time window is determined by physics, not by design choice	Control theory stability margin determines maximum communication delay
Structural completeness	Every required component must be present and accounted for	When elements are parts of a system and the question concerns whether the system is fully specified	Completeness is checkable by comparing the element list against the requirements	Every entity group has events; every event has constraints

Table C: Application Summary

Question	Domain Scope	Element Type	Constraint	Elements Enumerated	Survivors	Finding
What state changes can this OS undergo?	Within-domain (software)	Events	One-sentence test: single bounded state change	~311 events	~311 events (after split/merge)	Complete OS specification in four flat lists
What does information do?	Within-domain (information theory)	Information activities	Coverage by Shannon's theory	All information activities	Processing (the uncovered half)	The op as unit of processing; the missing symmetric theory
What representations preserve exact equality?	Within-domain (arithmetic)	Representation systems	Exact equality recovery after arbitrary operation chains	Scalar float, scalar rational, symbolic, triple	Triple [V, D, R]	The remainder slot is necessary for exact recovery in fixed-shape representation
What behavior can a software system exhibit?	Within-domain (software architecture)	Behavioral verbs	Can the behavior be expressed as guarded, staged, recorded, scoreable movement of quantities between addresses?	All behaviors across three test domains	One verb: movement of quantities	All behavior is the same behavior; new behavior is new rows, not new verbs

Question	Domain Scope	Element Type	Constraint	Elements Enumerated	Survivors	Finding
Is this substance food?	Within-domain (nutrition/physiology)	Ingestible substances	Four-question body-response rule (provide, load, sustained, occasional)	All ingestible substances	Frequency-dependent category assignment	Same substance can be food or not-food depending on intake pattern
What is the common structure in circular cross-section equations?	Cross-domain (nine engineering departments)	Terms in published equations	Algebraic decomposition into $F \times \beta \times d^2 \times Z$	Nine equations from nine departments	$\beta = \pi/4$ as universal geometric invariant	Nine departments derived the same operation under nine names
What carries coordination signals through biological tissue?	Cross-domain (neuroscience, physics)	Physical propagation mechanisms	Control theory timing inequality	Eight mechanisms from five disciplines	Electromagnetic field propagation	Classical carriers fail for demanding motor tasks; only EMF passes
What is the operational definition of mass?	Cross-domain (mechanics, GR, QCD, particle physics)	Departmental definitions of mass	Identity test: does the definition reduce to the definition of inertia?	Definitions from four departments	One property: inertia	Mass is inertia; the equivalence principle is a tautology

Question	Domain Scope	Element Type	Constraint	Elements Enumerated	Survivors	Finding
What produces the QED electron g-2?	Within-domain (quantum electrodynamics)	Input values to the derivation chain	Exact arithmetic — zero computational error contribution	32 values (12 rational coefficients, 5 transcendentals, 3 numerical, 4 SI constants, 2 measurements, 6 references)	22 structural + 2 measured → 4 derived CODATA values	Error propagation follows exact α-power scaling; arithmetic contributes zero error

---

Table D: Demand Matrix Template

A blank template the reader fills in for their own domain.

Element	Description	Measured Value	2 × Generous	4 × Extreme	Constraint	Result (Measured)	Result (Generous)	Result (Extreme)
						PASS / FAIL	PASS / FAIL	PASS / FAIL
						PASS / FAIL	PASS / FAIL	PASS / FAIL
						PASS / FAIL	PASS / FAIL	PASS / FAIL

Instructions for use:

1. Fill in the Element column with every element from Step 2.
2. Fill in the Measured Value column with the element's measured quantity relevant to the constraint.
3. Fill in the 2 × Generous column by doubling the element's performance (or halving its cost, whichever is more favorable to the element surviving).

4. Fill in the 4 × Extreme column by quadrupling the element's performance with no empirical basis.
5. Fill in the Constraint column with the bound from Step 3 (same for all rows).
6. For each cell in the Result columns, compare the element's value at that generosity level against the constraint. Record PASS, FAIL, or MARGINAL.
7. Read the completed matrix by row (how does each element perform?) and by column (what survives at each level?).

Table E: Generous Table Template

A blank template for constructing the generous table.

Parameter	Measured Value	Source	2 × Generous Value	Basis for Generosity	4 × Extreme Value	Basis for Extreme	Finding at Measured	Finding at Generous	Finding at Extreme

Instructions for use:

1. List every parameter whose exact value is uncertain.
2. Record the measured value and its source (publication, dataset, standard reference).
3. Set the generous value at 2 × the measured value in the direction most favorable to the alternative hypothesis.
4. State the basis for generosity — what undiscovered enhancement the generous value would require.
5. Set the extreme value at 4 × measured, with the basis stated as having no empirical support.
6. Re-run the elimination at each level.
7. Record the finding at each level.
8. If the finding holds at 4 × extreme, it is robust against any plausible shift in the parameter.

Table F: Common Failure Modes

Failure Mode	Symptom	Root Cause	Correction	Verification
Wrong element type	Constraint cannot be applied to the elements; elimination produces no result	The question and the element type are misaligned	Return to Step 1; restate the question until elements and constraint align	The constraint produces a clear pass/fail for every element
Incomplete enumeration	A later-discovered element survives the constraint and changes the finding	Elements were excluded by judgment rather than by constraint	Include everything; extend the enumeration when new elements appear	No known element in the domain is absent from the list
Debatable constraint	Readers disagree about whether specific elements pass or fail	The constraint depends on interpretation rather than measurement or logic	Replace with a constraint whose application is mechanical	Two independent readers applying the constraint to the same element get the same result
Fitted constraint	The finding is suspiciously clean; the constraint was adjusted after seeing results	The constraint was not stated before the elimination	Discard the finding; state the constraint in writing before re-eliminating	The constraint is documented with a timestamp or version prior to the elimination record
Subjective elimination	The investigator made judgment calls during Step 4	The constraint was inarguable in principle but insufficiently specific for mechanical application	Return to Step 3; tighten the constraint until application is binary	Every element's result can be derived by a reader who has only the element's data and the constraint
Unnamed survivors	The finding exists as a private list; no one else references or builds on it	Step 5 was skipped or performed informally	Name the surviving structure explicitly; write the name down	The finding can be communicated in one sentence using the name

Table G: Falsification Criteria

ID	Criterion	What It Looks Like	What It Means
F1	No structural differentiation from a previously known universal result	Every element passes or every element fails, and this was known before the enumeration	The method produced no new information in this application
F2	Incomplete enumeration artifact	A discovered element survives the constraint and changes the finding	The specific finding is invalid; Step 2 was violated; the method's requirement for exhaustive enumeration is confirmed
F3	Fitted constraint artifact	The constraint was adjusted to produce the desired result	The specific finding is circular; Step 3 was violated; the method's requirement for advance constraint statement is confirmed
F4	Correct execution produces a wrong finding	Complete enumeration, inarguable constraint, mechanical elimination — but the surviving structure is factually incorrect	The method's core claim is falsified: exhaustive enumeration plus inarguable constraint does not reliably produce structurally sound findings
F5	Alternative method matches UEM findings without exhaustive enumeration	A sampling or heuristic method produces the same findings with equal reliability and auditability	The exhaustive enumeration step is unnecessary overhead; the UEM is inefficient relative to the alternative

Table H: Within-Domain vs Cross-Domain Application

Property	Within-Domain	Cross-Domain
Elements come from	One department, one discipline, one system	Multiple departments, disciplines, or traditions
Step 1 (question)	Identical	Identical
Step 2 (enumerate)	Identical — list every element within the domain	Identical — list every element, including those from other domains
Step 3 (constraint)	Identical	Identical
Step 4 (eliminate)	Identical	Identical
Step 5 (name)	Identical	Identical
Primary output	Structural finding within the domain	Structural finding, potentially revealing shared structure across domains
Additional output	None specific to within-domain	Discovery that different domains share structure they did not know they shared
When to use	The question concerns one domain's internal structure	The question spans boundaries, or the enumeration naturally includes elements from multiple sources
Domain knowledge required	Expertise in the domain being investigated	Sufficient knowledge of each domain to enumerate its elements and verify the constraint's application
Difficulty	Standard — one domain to enumerate	Higher — multiple domains to enumerate; risk of incomplete enumeration increases with domain count

[**HOWL-INFO-18-2026**] The Universal Elimination Method. Github: https://github.com/ghowland/papers/tree/main/pape
INFO-18-2026

[**HOWL-INFO-14-2026**] Bits and Ops. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-
INFO-14-2026